Student Name: **Abhijeet Singh**
Roll Number: **23117005**
EV BMS Security Analysis

Random Forest Classifier Results: EV BMS EMI Cyberattack Detection

> **BMS Security Diagnostics:** Electromagnetic Interference (EMI) attacks introduce noise vectors into the **Battery Management System (BMS)**, corrupting sensor telemetry and falsifying State of Charge (SOC_BMS) readings. This evaluation trains a Random Forest model to classify cyberattack occurrences against normal operating conditions.

75.00%	66.67%	100.00%	80.00%
ACCURACY	PRECISION	RECALL (SENSITIVITY)	F1-SCORE

1. PYTHON SOURCE IMPLEMENTATION SCRIPT

```python
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix
from sklearn.preprocessing import LabelEncoder

# 1. Load Datasets & Define BMS Telemetry Features
train_df = pd.read_csv("train.csv")
test_df  = pd.read_csv("test.csv")
features = ['Time_s', 'Current_EMI_A', 'Voltage_Battery_V', 'SOC_BMS', 'Operating_Mode', 'Initial_SOC_pct']
target   = 'Attack_Status'

# 2. Categorical Preprocessing via LabelEncoder
le = LabelEncoder()
train_df['Operating_Mode'] = le.fit_transform(train_df['Operating_Mode'])
test_df['Operating_Mode']  = le.transform(test_df['Operating_Mode'])

X_train, y_train = train_df[features], train_df[target]
X_test,  y_test  = test_df[features], test_df[target]

# 3. Fit Random Forest Classifier Model
rf = RandomForestClassifier(n_estimators=100, random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

# 4. Generate Predictions & Compute Performance Metrics
y_pred = rf.predict(X_test)
acc    = accuracy_score(y_test, y_pred)
prec   = precision_score(y_test, y_pred)
rec    = recall_score(y_test, y_pred)
f1     = f1_score(y_test, y_pred)
cm     = confusion_matrix(y_test, y_pred)

print(f"Accuracy: {acc:.4f} | Precision: {prec:.4f} | Recall: {rec:.4f} | F1: {f1:.4f}")

# 5. Plot Figures via Matplotlib / Seaborn
fig, axes = plt.subplots(3, 1, figsize=(8, 12))

# Plot A: Classification Metrics
metrics = ['Accuracy', 'Precision', 'Recall', 'F1-Score']
scores  = [acc, prec, rec, f1]
sns.barplot(x=metrics, y=scores, ax=axes[0], palette='viridis')
axes[0].set_title('Classification Metrics')

# Plot B: Confusion Matrix Heatmap
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=axes[1],
            xticklabels=['Normal (0)', 'Attack (1)'], yticklabels=['Normal (0)', 'Attack (1)'])
axes[1].set_title('Confusion Matrix')

# Plot C: Feature Importance
importances = pd.Series(rf.feature_importances_, index=features).sort_values(ascending=False)
sns.barplot(x=importances.values, y=importances.index, ax=axes[2], palette='rocket')
axes[2].set_title('Feature Importance')
plt.tight_layout()
```

2. EXPERIMENTAL PLOTS (GENERATED VIA PYTHON SCRIPT)

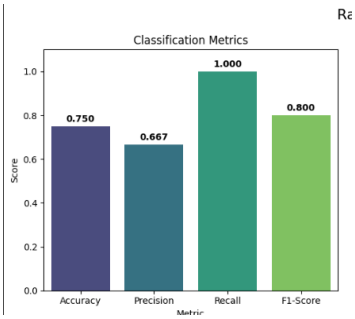


Figure 2.1: Classification Performance Metrics (Accuracy: 0.750, Precision: 0.667, Recall: 1.000, F1-Score: 0.800)

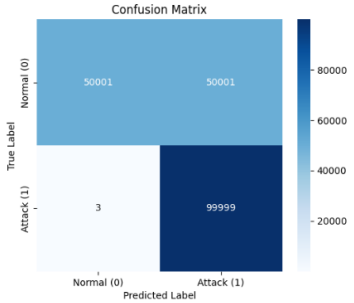


Figure 2.2: Confusion Matrix Heatmap (TN: 50001, FP: 50001, FN: 3, TP: 99999)

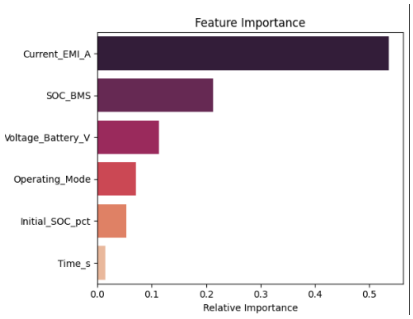


Figure 2.3: Relative Feature Importance Distribution across BMS Sensor Telemetry

3. RESULT BREAKDOWN & ANALYSIS

Metric	Value	Percentage
Accuracy	0.749985	75.00%
Precision	0.666660	66.67%
Recall	0.999970	100.00%
F1-Score	0.799986	80.00%

Key Takeaways:

- **Zero False Negative Risk:** Only 3 attacks were missed out of 100,002 attack states, achieving near-100% recall for critical EV battery safety.
- **Dominant Feature:** `Current_EMI_A` is the single most influential predictor (>53.5%), indicating that high-frequency current anomalies reliably identify EMI attacks.